

数据库系统 Database System

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr

关系数据库

介绍

- 关系数据库的结构
- 数据库架构
- 键
- 架构图
- 关系查询语言
- 关系代数

讲师关系示例

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	属性 (或列)
10101	Srinivasan	Comp. Sci.	65000	元组 (或行)
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

关系模式和实例

- $A_1, A_2, \dots, A_n$ 是 **属性**
- $R = (A_1, A_2, \dots, A_n)$ 是 **关系模式**

例子:

`instructor = (ID, name, dept_name, salary)`

- 关系**实例** r 在模式 **R** 上定义 用 **$r(R)$** 表示。
- 关系的当前值由**表**指定
- 元素 **t** 的关系 **r** 被称为**元组** 并由**一行表示**在表格中

属性

- 允许值集合称为域 属性
- 属性值（通常）要求是原子的；也就是说，不可分割
- 特殊价值 *null* 是每个域的成员。表示该值“未知”
- 空值导致许多操作的定义变得复杂

关系是无序的

- 元组的顺序无关紧要（元组可以按任意顺序存储）
 - 示例：具有无序元组的讲师关系

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

数据库模式/架构

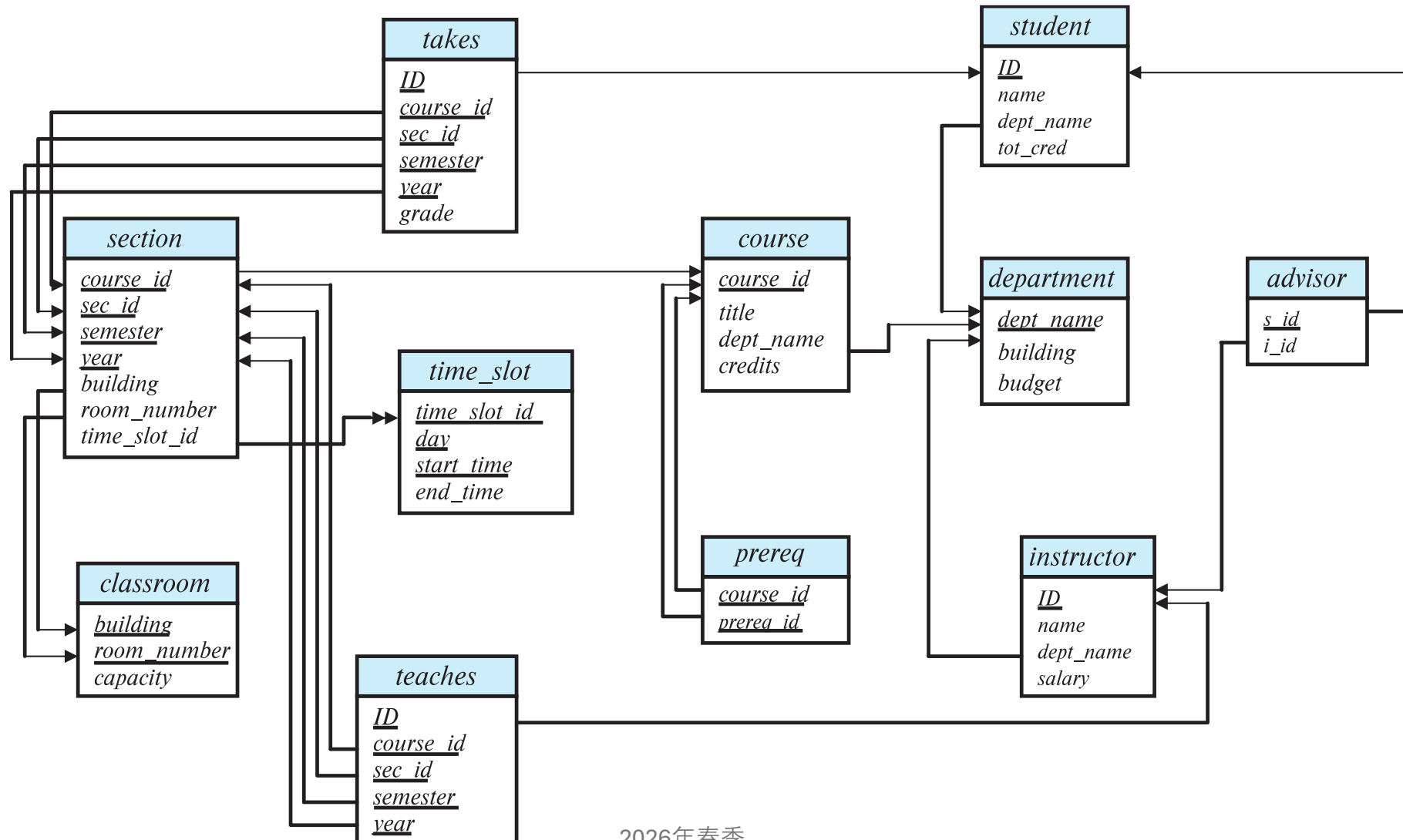
- **数据库模式**——
是数据库的**逻辑结构**。
- **数据库实例**——
是数据库中某一特定时刻的数
据**快照**。
- **例子：**
 - 模式：instructor (ID, name,
dept_name, salary)
 - 实例：

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

键

- 让 $K \subseteq R$
- 如果 K 的值足以识别每个可能关系 $r(R)$ 的唯一元组，则 K 是 R 的超级键
 - 例如：{ID} 和 {ID, name} 都是 instructor 的超键。
- 如果 K 最小，则超级键 K 是候选键
 - 例如：{ID} 是 Instructor 的候选键
- 候选键之一被选为主键。用下划线带指示。
 - 哪一个？
- 外键约束：一个关系中的值必须出现在另一个关系中
 - 引用关系：该关系具有主键。
 - 被引用关系：该关系具有另一个关系的主键。
 - 例如：再 讲师关系 中的 *dept_name* 是引用 部门 的外键

大学数据库架构图



关系查询语言

- 过程式 与 非过程式或声明式
- “纯”语言：
 - 关系代数
 - 元组关系演算
 - 领域关系演算
- 上述 3 种纯语言在计算能力上是相当的
- 本次课 我们将集中讨论 关系代数
 - 非图灵机等价物
 - 6个基本操作组成

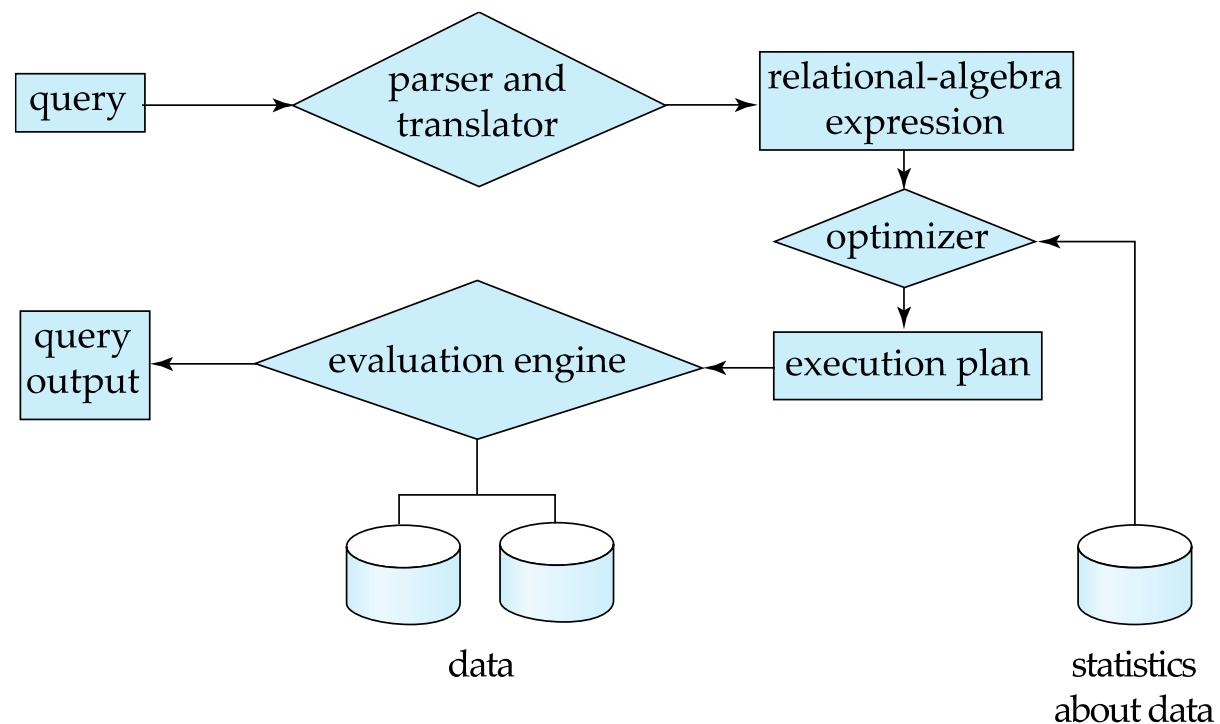
关系代数

- 一种过程语言，由一组操作组成，这些操作以一个或两个关系作为输入，并产生一个新的关系作为结果。
- 六个基本运算符
 - 选择： σ (sigma)
 - 投影： π (Pi)
 - 并集： $\cup$ (交集： $\cap$)
 - 差集： $-$
 - 笛卡尔积： $\times$
 - 重命名： ρ (Rho)

查询处理

查询处理

- 1. 解析与翻译
- 2. 优化
- 3. 评估



选择操作

- 选择操作选择满足给定谓词的元组。

- 符号: $\sigma_p(r)$

- p 被称为选择谓词 (作为过滤器/条件)

- 例如: 选择 讲师关系 中 讲师 属于“物理”系的元组。
- 询问

$\sigma_{dept_name = "Physics"}(instructor)$

- 结果

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

选择操作 (续)

- 我们允许使用

=、≠、>、≥、<、≤

在选择谓词中。

- 我们可以使用连接词将几个谓词组合成一个更大的谓词：

∧ (和) , ∨ (或) , ¬ (否)

- 例如：查找薪水高于 90,000 美元的物理学讲师，我们写为：

$\sigma_{dept_name="Physics" \wedge salary > 90,000} (instructor)$

- 选择谓词可能包括两个属性之间的比较。
 - 例如，查找所有名称与建筑物名称相同的部门：

$\sigma_{dept_name=building} (department)$

选择操作示例

$R(a_id, b_id)$

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

$\sigma_{a_id='a2' \wedge b_id > 102}(R)$

a_id	b_id
a2	103

```
SELECT * FROM R
WHERE a_id='a2' AND b_id>102;
```

$R(a_id, b_id)$

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

$\sigma_{a_id='a2' \wedge b_id > 102}(R)$

a_id	b_id
a2	103

```
SELECT * FROM R
WHERE a_id='a2' AND b_id>102;
```


投影操作

- 返回其参数关系的一元运算，省略某些属性。

- 符号： $\pi_{A_1, A_2, A_3 \dots A_k}(r)$

其中 $A_1, A_2, \dots, A_k$ 是属性名称， r 是关系名称。

- 通过擦除未列出的列得到的 k 列关系
- 由于关系是集合，因此从结果中删除重复的行

投射操作示例

- 示例：消除 *instructor* 的 *dept_name* 属性

- 询问：

$\pi_{ID, name, salary}(instructor)$

- 结果：

```
SELECT ID, name, salary  
FROM instructor;
```

ID	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

关系运算的构造

- 关系代数运算的结果是关系，因此关系代数运算可以组合成关系代数表达式。
- 考虑以下查询——查找物理系所有讲师的姓名。

$\pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$

- 我们不是将关系的名称作为投影操作的参数，而是给出一个计算关系的表达式。

笛卡尔积运算

- 笛卡尔积运算（用**X表示**）允许我们**组合任意两个关系中的信息**。
 - 例如：instructor 和 teaches 的关系的笛卡尔积写为：
instructor X teaches
- **每对可能的元组**中构建一个结果元组：一个来自 instructor 关系，一个来自 teaches 关系（参见下一张幻灯片）
- 由于讲师**ID**出现在两种关系中，我们通过将属性附加到该属性最初来自的关系的名称来区分这些属性。
 - instructor.ID
 - teaches.ID

instructor **X** *teaches* 表

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

连接 (join) 操作

- 笛卡尔积

instructor X teaches

将每个 *instructor* 元组与每个 *teaches* 元组关联起来。

- 结果行中的大多数信息都与未教授特定课程的教师有关。
- 与讲师及其教授的课程相关的“*instructor X teaches*”的元组，我们这样写：

$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$

- 与讲师及其教授的课程有关的“*讲师X教授*”的元组。
- 表达式的结果显示在下一张幻灯片中

连接操作 (续)

- 表为:

$\sigma_{instructor.id = teaches.id}(instructor \times teaches)$

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

连接操作 (续)

- 连接操作允许我们将选择操作和笛卡尔积运算合并为单一运算。
- 考虑关系 $r(R)$ 和 $s(S)$
- 令“theta”为模式 R “union” S 中属性的谓词。连接操作 $r \bowtie_{\theta} s$ 定义如下:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

- 因此

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

- 可以等效地写成

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

联合操作

- 联合运算允许我们将两个关系结合起来：符号： $r \cup s$
- 对于 $r \cup s$ 是有效的，必须满足。
 1. r, s 必须具有相同的元数（相同数量的属性）
 2. 属性域必须兼容（例如：第二个 r 列处理的值类型与 s 的第二列）
- 示例：查找 2017 年秋季学期、2018 年春季学期或两个学期教授的所有课程

$\pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$

$\pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$

联合操作 (续)

- 结果:

$\pi_{course_id}(\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$

$\pi_{course_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

集合交运算

- 集合交集运算使我们能够找到同时存在于两个输入关系中的元组。
- 符号: $r \cap s$
- 假设:
 - r , s 具有相同的元数
 - r 和 s 的属性兼容
- 示例: 查找 2017 年秋季和 2018 年春季学期教授的所有课程的集合。

$\pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cap$

$\pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$

- 结果

course_id
CS-101

集合差集运算

- 集合差分运算使我们能够找到存在于一个关系中但不存在于另一个关系中的元组。
- 符号 $r - s$
- 兼容关系之间必须采取一定的差异。
 - r 和 s 必须具有相同的元数
 - r 和 s 的属性域必须兼容
- 示例：查找 2017 年秋季学期开设但 2018 年春季学期未开设的所有课程

$\pi_{course_id}(\sigma_{semester="Fall" \wedge year=2017}(section)) -$

$\pi_{course_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$

<i>course_id</i>
CS-347
PHY-101

赋值操作

- 有时通过将关系代数表达式的各个部分分配给临时关系变量来编写关系代数表达式会很方便。
- **赋值运算**用 $\leftarrow$ 表示，其工作方式类似于编程语言中的赋值。
- 示例：查找“物理”和音乐系的所有讲师。

$Physics \leftarrow \sigma_{dept_name="Physics"}(instructor)$

$Music \leftarrow \sigma_{dept_name="Music"}(instructor)$

$Physics \cup Music$

使用赋值操作，可以将查询写成一个顺序程序，该程序由一系列赋值操作组成，后跟一个表达式，该表达式的值显示为查询的结果。

重命名操作

- 关系代数表达式的结果没有可供引用的名称。为此，我们提供了重命名运算符 ρ
- 表达式：

$$\rho_x (E)$$

返回 名称为 x 的表达式

- 重命名操作的另一种形式：

$$\rho_{x(A1,A2,.. An)} (E)$$

等效查询

- 在关系代数中，编写查询的方法不止一种。
- 示例：查找物理系工资高于 90,000 的教师所讲授的课程信息
- 查询 1

$\sigma_{dept_name = \text{"Physics"} \wedge salary > 90,000} (instructor)$

- 查询 2

$\sigma_{dept_name = \text{"Physics"}} (\sigma_{salary > 90,000} (instructor))$

- 这两个查询并不相同；但它们是等效的——它们在任何数据库上都会给出相同的结果。

等效查询

- 在关系代数中，编写查询的方法不止一种。
- 示例：查找物理系教师教授的课程信息
- 查询 1

$\sigma_{dept_name="Physics"}(instructor \bowtie_{instructor.ID = teaches.ID} teaches)$

- 查询 2

$(\sigma_{dept_name="Physics"}(instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$

- 这两个查询并不相同；但它们是等效的——它们在任何数据库上都会给出相同的结果。

SQL简介

大纲

- SQL查询语言概述
- SQL数据定义
- SQL 查询的基本查询结构
- 附加基本操作
- 集合运算
- 空值
- 聚合函数
- 嵌套子查询
- 数据库修改

历史

- IBM Sequel 语言是 IBM 圣何塞研究实验室的 System R 项目的一部分开发的
- 重命名结构化查询语言 (SQL)
- ANSI 和 ISO 标准 SQL:
 - SQL-86
 - SQL-89
 - **SQL-92**
 - SQL:1999 (语言名称符合 Y2K 标准!)
 - SQL: 2003
 - SQL: 2008 → 截断、花式排序 (Truncation, Fancy Sorting)
 - SQL: 2011 → 时态数据库、流水线 DML (Temporal DBs, Pipelined DML)
 - SQL: 2016 → JSON、多态表 (JSON, Polymorphic tables)
 - SQL: 2023 → 属性图查询、多维数组 (Property Graph Queries, Muti-Dim. Arrays)
- 商业系统提供大多数 (如果不是全部) **SQL-92** 功能, 以及来自后续标准的不同功能集和特殊专有功能。
 - 这里的示例可能并不适用于您的特定系统。

SQL 部分

- **DML**——提供从数据库查询信息以及在数据库中插入、删除和修改元组的能力。
- **完整性**——DDL 包括用于指定**完整性约束的命令**。
- **视图定义**——DDL 包括**定义视图的命令**。
- **事务控制**——包括指定事务开始和结束的命令。
- **嵌入式 SQL 和动态 SQL**——定义如何将 SQL 语句嵌入通用编程语言中。
- **授权**——包括指定关系和视图访问权限的命令。

数据定义语言

- **SQL 数据定义语言 (DDL)** 允许指定有关关系的信息，包括：
 - 每个关系的**模式**。
 - 与每个属性关联的**值的类型**。
 - 完整性约束
 - 为每个关系维护的**索引集**。
 - 每个关系的安全和授权信息。
 - 磁盘上每个关系的物理存储结构。

SQL 中的域类型

- **char(n)** 固定长度字符串，用户指定长度 n 。
- **varchar(n)** 可变长度字符串，最大长度由用户指定 n 。
- **int** 整数（与机器相关的整数的有限子集）。
- **smallint** 小整数（整数域类型的机器相关子集）。
- **numeric(p,d)** 定点数，用户指定精度为 p 位，小数点右侧有 d 位。（例如，**numeric (3,1)** 允许精确存储 44.5，但不能存储 444.5 或 0.32）
- **real, double precision** 浮点数和双精度浮点数，具有机器相关的精度。
- **float(n)** 浮点数，用户指定的精度至少为 n 位。
- 稍后将介绍更多内容。

数据库系统环境

- SQLite3



- 下载: <https://www.sqlite.org/2025/sqlite-tools-win-x64-3500400.zip>
(6.13 MiB) by 2025/09/07

- MariaDB



- 下载: https://mariadb.org/download/?t=mariadb&p=mariadb&r=12.0.2&os=windows&cpu=x86_64&pkg=msi&mirror=blendbyte (81.2 MB)

- PostgreSQL



PostgreSQL

- 下载: <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads> (347MB)

SQLite3 命令行界面：教程

- 一个名为 `sqlite3`（Windows 上为 `sqlite3.exe`）的简单命令行程程序，允许用户手动输入并执行针对 SQLite 数据库或 ZIP 压缩包的 SQL 语句。
- 在命令提示符下输入“`sqlite3`”，并可选择输入包含 SQLite 数据库（或 ZIP 压缩包）的文件名。
- 如果指定的文件不存在，则会自动创建一个具有指定名称的新数据库文件。
- 如果未在命令行中指定数据库文件，则会创建一个临时数据库，并在“`sqlite3`”程序退出时自动删除该数据库。

创建表构造

- SQL 关系使用 **创建表** 命令:

```
create table r  
  
(A1 D1, A2 D2, ..., An Dn,  
(integrity-constraint1),  
...,  
(integrity-constraintk))
```

- *r* 是关系的名称
- 每个 *A*_{*i*} 是在 *r* 模式中的一个属性名称
- *D*_{*i*} 是属性 *A*_{*i*} 域的数据类型
- 例子:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name    varchar(20),  
    salary       numeric(8,2)  
);
```

创建表中的完整性约束

- 完整性约束的类型
 - primary key ($A_1, \dots, A_n$)
 - foreign key ($A_m, \dots, A_n$) references r
 - 非空
- SQL 阻止任何违反完整性约束的数据库更新。
- 例子:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name    varchar(20),  
    salary       numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department  
);
```

还有一些关系定义

```
create table student (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    tot_cred    numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department  
);  
  
create table takes (  
    ID          varchar(5),  
    course_id   varchar(8),  
    sec_id      varchar(8),  
    semester    varchar(6),  
    year        numeric(4,0),  
    grade       varchar(2),  
    primary key (ID, course_id, sec_id, semester, year),  
    foreign key (ID) references student, foreign key (course_id, sec_id,  
semester, year) references section  
);
```

还有更多

```
create table course (  
    course_id    varchar(8),  
    title        varchar(50),  
    dept_name    varchar(20),  
    credits      numeric(2,0),  
    primary key  (course_id),  
    foreign key  (dept_name) references department  
);
```

表格更新

- 插入

```
insert into instructor values ('10211', 'Smith', 'Biology', 66000);
```

- 删除

- 从 学生 关系中删除所有元组

```
delete from student;
```

- 删除表

```
drop table r;
```

- 改变

- ```
alter table r add A D;
```

- 其中A是要添加到关系r 的属性的名称， D是A
- 关系中所有现有的元组都被分配为新属性的值null 。

- ```
alter table r drop A;
```

- 其中A是关系r的属性名称
- 删除许多数据库不支持的属性。

基本查询结构

- 典型的 SQL 查询具有以下形式：

```
select A1, A2, ..., An  
from r1, r2, ..., rm  
where P
```

- A_i 表示一个属性
- r_i 表示一种关系
- P 是一个谓词。
- SQL 查询的结果是一个关系。

select子句

- **select**子句列出了查询结果中所需的**属性**
 - 对应于关系代数的**投影运算**
- 例如：查找所有讲师的姓名：

```
select name  
from instructor;
```

- 注意：SQL 名称**不区分**大小写（即，可以使用大写或小写字母。）
 - 例如， **Name** $\equiv$ **NAME** $\equiv$ **name**
 - 无论我们使用粗体字体，有些人都会使用大写字母。

select 子句 (续)

- SQL 允许关系和查询结果中**存在重复**。
- 要强制消除重复项，请插入关键字**distinct** 选择后。
- 查找所有讲师的部门名称，并删除重复项

```
select distinct dept_name  
from instructor;
```

- 关键字**all** 指定不应删除重复项。

```
select all dept_name  
from instructor;
```

dept_name

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

select 子句 (续)

- select 子句中的星号表示“所有属性”

```
select * from instructor;
```

- 属性可以是没有from子句的文字

```
select '437'
```

- 结果是一个包含一列和一行且值为“437”的表格
- 可以使用以下命令为列命名:

```
select '437' as F00
```

- 属性可以是带有from子句的文字

```
select 'A' as F00  
from instructor
```

- *N*行 (讲师表中的元组数) 的表, 每行都有值“A”

select 子句 (续)

- 选择子句可以包含涉及运算 +、-、* 和 / 的算术表达式，以及对元组的常量或属性进行运算。

- 查询：

```
select ID, name, salary/12  
from instructor;
```

将返回与 *Instructor* 关系相同的关系，只是属性 *salary* 的值要除以 12。

- 使用 **as** 重命名 “*s salary/12*” 条款：

```
select ID, name, salary/12 as monthly_salary  
from instructor;
```

where子句

- where 子句指定结果必须满足的**条件**
 - 对应于关系代数的选择谓词
- 查找计算机科学系的所有讲师

```
select name  
from instructor  
where dept_name = 'Comp. Sci.';
```

- SQL 允许使用逻辑连接词 **和、或、和非**
- 逻辑连接词的操作数可以是涉及比较运算符 <、<=、>、>=、= 和 <> 的表达式
- 比较可以应用于算术表达式的结果
- 查找计算机科学系所有薪水 > 70000 的讲师

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 70000;
```

<i>name</i>
Katz
Brandt

from子句

- from 子句列出了查询中涉及的**关系**
 - 对应于关系代数的笛卡尔积运算。
- 查找**笛卡尔积**讲师 X 教授的课程

```
select * from instructor, teaches
```

- 生成每一个可能的导师-教师对，包含来自两个关系的所有属性。
- 对于常见属性（例如，*ID*），结果表中的属性使用关系名称重命名（例如，*instructor.ID*）
- 笛卡尔积直接使用时不是很有用，但与 where 子句条件（关系代数中的选择运算）结合使用时很有用。

示例

- 查找教授过某门课程的所有教师的姓名以及课程 ID

```
select name, course_id
from instructor , teaches
where instructor.ID = teaches.ID;
```

- 查找艺术系所有教授过某门课程的教师姓名以及课程编号

```
select name, course_id
from instructor , teaches
where instructor.ID = teaches.ID
      and instructor. dept_name = 'Art';
```

<i>name</i>	<i>course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

重命名操作

- SQL 允许使用as子句重命名关系和属性：
old-name as new-name

- 找出所有薪水高于
“计算机科学”领域某些讲师的讲师姓名。

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept_name = 'Comp. Sci.';
```

- 关键字as是可选的，可以省略
instructor as T $\equiv$ *instructor T*

自连接示例

- *emp-super*关系 (职员-主管关系)

<i>person</i>	<i>supervisor</i>
Bob	Alice
Mary	Susan
Alice	David
David	Mary

- 找到“Bob”的主管
- 找到“Bob”的主管的主管
- 你能找到“鲍勃”的所有主管（直接和间接）吗？

字符串操作

- SQL 包含一个字符串匹配运算符，用于对字符串进行比较。like 运算符使用由两个特殊字符描述的模式：
 - 百分号（%）。匹配任何子字符串。
 - 下划线（_）。匹配任何单一字符。
- 查找所有教师的姓名中包含‘dar’子字符串。

```
select name  
from instructor  
where name like '%dar%'
```

- 匹配字符串“100%”
like '100\%' escape '\'

在上文中我们使用反斜杠 (\) 作为转义字符。

字符串操作（续）

- 模式区分大小写。
- 模式匹配示例：
 - 'Intro%' 匹配以“Intro”开头的任何字符串。
 - '%Comp%' 与任何包含“Comp”作为子字符串的字符串匹配。
 - '___' 匹配任意由三个字符组成的字符串。
 - '___%' 匹配至少三个字符的任何字符串。
- SQL 支持多种字符串操作，例如
 - 连接（使用“||”）
 - 从大写转换为小写（反之亦然）
 - 查找字符串长度、提取子字符串等。

元组的显示顺序

- 按字母顺序列出所有讲师的姓名

```
select distinct name  
from instructor  
order by name;
```

- 我们可以指定 **desc** 对于每个属性按降序排列，也可以按 **asc** 升序排列；升序是默认顺序。

- 例如： **order by name desc;**

- 可以按多个属性排序

- 例如： **order by dept_name, name;**

Where 子句谓词

- SQL包含比较运算符, **between**
- 示例：查找薪水在 90,000 美元到 100,000 美元之间的所有讲师的姓名（即 $\geq 90,000$ 美元和 $\leq 100,000$ 美元）

```
select name  
from instructor  
where salary between 90000 and 100000;
```

- 元组比较

```
select name, course_id  
from instructor, teaches  
where (instructor.ID, dept_name) = (teaches.ID,  
'Biology');
```

集合运算

- 查找 2017 年秋季或 2018 年春季开设的课程

```
(select course_id from section where sem = 'Fall' and year = 2017)  
union  
(select course_id from section where sem = 'Spring' and year = 2018)
```

- 查找 2017 年秋季和 2018 年春季开设的课程

```
(select course_id from section where sem = 'Fall' and year = 2017)  
intersect  
(select course_id from section where sem = 'Spring' and year = 2018)
```

- 查找 2017 年秋季开设但 2018 年春季未开设的课程

```
(select course_id from section where sem = 'Fall' and year = 2017)  
except  
(select course_id from section where sem = 'Spring' and year = 2018)
```

集合运算 (续)

- 集合运算**并集**、**交集**和**除集**
 - 上述每个操作都会自动消除重复项
- 要保留所有重复项，请使用
 - **union all**
 - **intersect all**
 - **except all**

空值 (Null)

- 元组的某些属性可能具有空值，用null表示
- null表示未知值或不存在值。
- null 的算术表达式的结果都是null
 - 例如：5 + null返回null
- 谓词为空可用于检查空值。
 - 示例：查找所有薪水为 null 的讲师。

```
select name  
from instructor  
where salary is null
```

- 如果应用谓词的值不为空，则谓词不为空。

空值 (续)

- 任何涉及空值的比较结果视为unknown（谓词is null和is not null除外）。
 - 例如: $5 < \text{null}$ 或者 $\text{null} \neq \text{null}$ 或者 $\text{null} = \text{null}$
- where子句中的谓词可以涉及布尔运算（与、或、非）；因此需要扩展布尔运算的定义以处理未知值。
 - 且:

$(\text{true and unknown})$	$= \text{unknown},$
$(\text{false and unknown})$	$= \text{false},$
$(\text{unknown and unknown})$	$= \text{unknown}$
 - 或:

(unknown or true)	$= \text{true},$
$(\text{unknown or false})$	$= \text{unknown}$
$(\text{unknown or unknown})$	$= \text{unknown}$
- where子句谓词的结果为未知

聚合函数

- 这些函数对关系列的多值集进行操作，并返回一个值

avg: 平均值

min: 最小值

max: 最大值

sum: 值的总和

count: 值的数量

聚合函数示例

- 查找计算机科学系教师的平均工资

```
select avg (salary)
from instructor
where dept_name= 'Comp. Sci.';
```

- 查找 2018 年春季学期教授某门课程的教师总数

```
select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2018;
```

- 课程关系中的元组数量

```
select count (*) from course;
```

聚合函数-分组

- 查找各部门讲师的平均工资

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name;
```

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

聚合 (续)

- 聚合函数之外的select子句中的属性必须出现在group by列表中

```
select dept_name, ID, avg (salary)
from instructor
group by dept_name, ID;
```

聚合函数 – **Having** 子句

- 查找平均工资大于42000的所有部门的名称和平均工资

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name  
having avg (salary) > 42000;
```

- 注意： **having**子句中的谓词在组形成后应用，而**where**子句中的谓词在组形成之前应用

嵌套子查询

- SQL 提供了一种嵌套子查询的机制。子查询是一个嵌套在另一个查询中的select-from-where表达式。
- 嵌套可以在以下 SQL 查询中完成

```
select A1, A2, ..., An  
from r1, r2, ..., rm  
where P;
```

如下：

- From 子句： r_i 可以用任何有效的子查询替换
- Where 子句： P 可以用以下形式的表达式替换：
 $B < operation > (\text{子查询})$
 B 是稍后要定义的属性和<operation>。
- Select 子句：
 A_i 替换为生成单个值的子查询。

集合的成员

Set membership

集合的成员

- 查找 2017 年秋季和 2018 年春季提供的课程

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and course_id in
    (select course_id from section
     where semester = 'Spring' and year= 2018);
```

- 查找 2017 年秋季提供但 2018 年春季不提供的课程

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and course_id not in
    (select course_id from section
     where semester = 'Spring' and year= 2018);
```

集合的成员 (续)

- 说出所有名字既不是“莫扎特”也不是“爱因斯坦”的老师的名字

```
select distinct name  
from instructor  
where name not in ('Mozart', 'Einstein');
```

- ID为10101 的教师所讲授课程的学生总数 (`distinct`) 。

```
select count (distinct ID)  
from takes  
where (course_id, sec_id, semester, year) in  
      (select course_id, sec_id, semester, year from teaches  
       where teaches.ID= 10101);
```

- 注意：上面的查询可以用更简单的方式编写。
上面的公式只是为了说明 SQL 的特性。

集合比较

Set Comparison

集合比较——“**some**”子句

- 查找薪水高于生物系某些（至少一位）讲师的讲师姓名。

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept name = 'Biology';
```

- 使用 > **some**子句的相同查询

```
select name  
from instructor  
where salary > some (select salary from instructor  
                      where dept_name = 'Biology');
```

“some”条款

- $F <\text{comp}> \text{some } r \Leftrightarrow \exists t \in r \text{ 使得 } (F <\text{comp}> t)$
其中 $<\text{comp}>$ 可以是: $<, \leq, >, =, \neq$

$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$ (读作: 5 < some tuple in the relation)

$(5 < \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$

$(5 = \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$

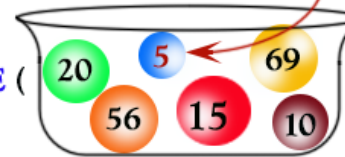
$(5 \neq \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$ (因为 $0 \neq 5$)

$(= \text{some}) \equiv \text{in}$ /
However, $(\neq \text{some}) \equiv \text{not in}$

SOME操作符

> SOME means greater than at least one value,
that is, greater than the minimum.

WHERE 70 > SOME (



So >SOME (20,56,5,15,69,10)
means greater than 5.

So 70 > 5 is true, and data returns.

< SOME means less than at least one value,
that is, less than the maximum.

WHERE 70 < SOME (

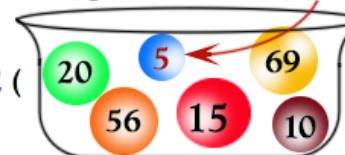


So <SOME (20,56,5,15,69,10)
means less than 69.

So 70 < 69 is false, and no data returns.

>SOME means greater than at least one value,
that is, greater than the minimum.

WHERE 4 > SOME (

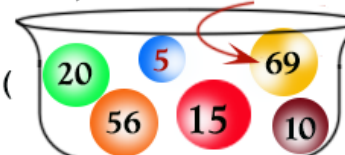


So >SOME (20,56,5,15,69,10)
means greater than 5.

So 4 > 5 is false, and no data returns.

< SOME means less than at least one value,
that is, less than the maximum.

WHERE 4 < SOME (



So <SOME (20,56,5,15,69,10)
means less than 69.

So 4 < 69 is true, and data returns.

© w3resource.com

集合比较 – “all”子句

- 查找所有工资高于生物系所有教师工资的教师姓名。

```
select name  
from instructor  
where salary > all (select salary from instructor  
                    where dept_name = 'Biology');
```

“all”条款定义

- $F < \text{comp} > \text{所有 } r \Leftrightarrow \forall t \in r (F < \text{comp} > t)$

$$(5 < \text{all} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \text{all} \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all} \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all} \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$$

$(\neq \text{all}) \equiv \text{not in}$
However, $(= \text{all}) \not\equiv \text{in}$

ALL操作符

>ALL means greater than the biggest value,
that is, greater than the maximum.

WHERE 70 > ALL (20 5 69 56 15 10);

So >ALL (20,56,5,15,69,10) means greater than 69.

So 70 > 69 is true, and data returns.

<ALL means less than the smallest value,
that is, less than the minimum

WHERE 70 < ALL (20 5 69 56 15 10);

So <ALL (20,56,5,15,69,10) means less than 5.

So 70 < 5 is false, and no data returns.

>ALL means greater than the biggest value,
that is, greater than the maximum.

WHERE 4 > ALL (20 5 69 56 15 10);

So >ALL (20,56,5,15,69,10) means greater than 69.

So 4 > 69 is false, and no data returns.

<ALL means less than the smallest value,
that is, less than the minimum.

WHERE 4 < ALL (20 5 69 56 15 10);

So <ALL (20,56,5,15,69,10) means less than 5.

So 4 < 5 is true, and data returns.

© w3resource.com

测试空关系

- 如果参数子查询非空，则exists构造返回值true。
- 存在 $r \Leftrightarrow r \neq \emptyset$
- 不存在 $r \Leftrightarrow r = \emptyset$

使用“exists”子句

- 指定查询“查找 2017 年秋季学期和 2018 年春季学期教授的所有课程”的另一种方式

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
              and S.course_id = T.course_id);
```

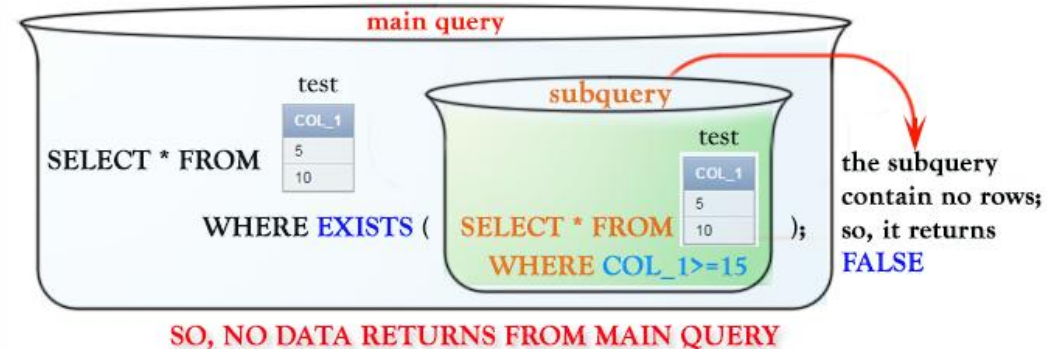
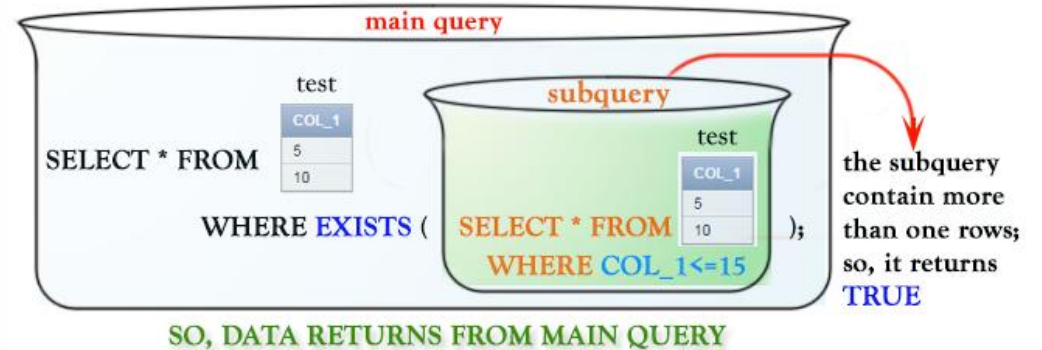
- 相关名称- 外部查询中的变量 S
- 相关子查询——内部查询

EXISTS 操作符

WHERE EXISTS (**subquery**);

EXISTS operator

- EXISTS is a Comparison operator
- Used in WHERE clause to validate an "IT EXISTS" condition.
- EXISTS will tell you whether a query returned any results.
- Returns a BOOLEAN, (TRUE or FALSE).
- Returns TRUE if a subquery contains any rows.



© w3resource.com

使用 “not exists” 子句

- 查找所有修过生物学系提供的所有课程的学生。

```
select distinct S.ID, S.name
from student as S
where not exists ( (select course_id from course
                    where dept_name = 'Biology')
                  except
                  (select T.course_id from takes as T
                   where S.ID = T.ID));
```

- 第一个嵌套查询列出了生物学提供的所有课程
 - 第二个嵌套查询列出了特定学生参加的所有课程
- 注意 $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
- 注意：不能使用 = all 及其变体编写此查询

测试是否存在重复元组

- **unique 构造**测试子查询的结果中是否有任何重复的元组。
- 如果给定的子查询不包含重复项，则 **unique 构造**的计算结果为“真”。
- 查找 2017 年最多开设一次的所有课程

```
select T.course_id
from course as T
where unique ( select R.course_id
               from section as R
               where T.course_id= R.course_id
                 and R.year = 2017);
```

From 子句中的子查询

From子句中的子查询

- from子句中使用子查询表达式
- 查找平均工资高于 42,000 美元的部门的平均教师工资。”

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```

- 请注意，我们不需要使用having子句
- 上述查询的另一种编写方式

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
      as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

With 子句

- **with**子句提供了一种定义临时关系的方法，该临时关系的定义仅适用于使用**with** 的查询子句发生。
- 查找所有预算最高的部门

```
with max_budget(value) as (select max(budget)
                             from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

使用 With 子句的复杂查询

- 查找所有总工资大于所有部门总工资平均值的部门

```
with dept_total(dept_name, value) as
    (select dept_name, sum(salary)
     from instructor
     group by dept_name),
dept_total_avg(value) as
    (select avg(value)
     from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```


标量子查询

- 标量子查询用于需要单个值的情况
- 列出所有部门以及每个部门的讲师人数

```
select dept_name, ( select count(*)  
                    from instructor  
                    where department.dept_name = instructor.dept_name)  
          as num_instructors  
from department;
```

- 如果子查询返回多个结果元组，则会出现运行时错误

数据库修改

- 从给定关系中**删除**元组。
- 将新元组**插入**给定关系
- **更新**给定关系中某些元组的值

删除

- 删除所有讲师

```
delete from instructor;
```

- 删除财务部门的所有讲师

```
delete from instructor  
where dept_name= 'Finance';
```

- 删除与位于Watson大楼的部门相关的讲师关系中的所有元组。

```
delete from instructor  
where dept_name in (select dept_name  
                    from department  
                    where building = 'Watson');
```

删除 (续)

- 删除所有工资低于讲师平均工资的讲师

```
delete from instructor  
where salary < (select avg (salary)  
                from instructor);
```

- 问题：当从 *instructor* 中删除元组时，平均工资会发生变化
- SQL中使用的解决方案：
 - 首先，计算平均值（工资）并找到所有要删除的元组
 - 接下来，删除上面找到的所有元组（不重新计算平均值或重新测试元组）

插入

- 课程添加新元组

```
insert into course values ('CS-437', 'Database  
Systems', 'Comp. Sci.', 4);
```

- 或等价地

```
insert into course (course_id, title, dept_name, credits)  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- 使用*tot_creds*为学生添加一个新元组 设置为空

```
insert into student  
values ('3003', 'Green', 'Finance', null);
```

插入 (续)

- 让音乐系每个修满 144 个以上学分的学生成为音乐系的讲师，年薪 18,000 美元。

```
insert into instructor
  select ID, name, dept_name, 18000
  from student
  where dept_name = 'Music' and total_cred > 144;
```

- 在将任何结果插入到关系中之前，会先对 select from where 语句进行全面评估。

否则查询如下

```
insert into table1 select * from table1;
```

会引起问题

更新

- 给所有讲师加薪 5%

```
update instructor set salary = salary * 1.05;
```

- 低于7万 的教员加薪5%

```
update instructor  
  set salary = salary * 1.05  
  where salary < 70000;
```

- 给工资低于平均水平的讲师加薪 5%

```
update instructor  
  set salary = salary * 1.05  
  where salary < (select avg (salary)  
                  from instructor);
```

更新 (续)

- 将年薪超过 10 万美元的讲师的工资提高 3%，其他讲师的工资提高 5%
- 写两个更新语句：

```
update instructor  
  set salary = salary * 1.03  
  where salary > 100000;
```

```
update instructor  
  set salary = salary * 1.05  
  where salary <= 100000;
```

- 顺序很重要
- case语句可以做得更好（下一张幻灯片）

条件更新的 **case** 语句

- 与之前相同的查询，但使用 **case** 语句

```
update instructor
  set salary =
    case
      when salary <= 100000 then salary * 1.05
      else salary * 1.03
    end
```

使用标量子查询进行更新

- 重新计算并更新所有学生的tot_creds值

```
update student S
  set tot_cred = (select sum(credits)
                  from takes, course
                  where takes.course_id = course.course_id
                     and S.ID = takes.ID
                     and takes.grade <> 'F'
                     and takes.grade is not null);
```

- 对于未参加任何课程的学生，将tot_creds设置为 null
 - 不要使用sum(credits)，而用：

```
update student S
  set tot_cred =
    case
      when sum(credits) is not null then sum(credits)
      else 0
    end
```

Database System

Thank you!

Q/A?

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr